

FCA Constrained Clustering

Klara M. Gutekunst^[0009-0002-6416-7873], Dominik
Dürrschnabel^[0000-0002-0855-4185], and Stefan Schneider^[TODO]

¹ Deep Semantic Learning Group, University of Kassel, Germany

² Interdisciplinary Research Center for Information System Design,
University of Kassel, Germany

`klara.gutekunst@uni-kassel.de`
`{duerrschnabel}@cs.uni-kassel.de`

Abstract. This is an abstract.

Keywords: Clustering · FCA · NLP · Text Mining · Hierarchical Topic
Representation · Semantic Analysis.

1 Introduction

2 Related Work



3 Foundations

Constraints Constraints model prior knowledge about an optimum [?].

Hierarchical clustering The purpose of clustering is to structure data without any class labels. This is especially useful in real world applications since data often lacks labels and the annotation process is expensive. Partitional clustering groups data points into clusters based on a similarity measure. Hierarchical clustering adds another level of usefulness by grouping the data on different levels of granularity [?]. Hence, partitional clustering is a special case of hierarchical clustering with one level of granularity.

Clustering algorithms group data points based on different criteria into clusters. The overall goal is to minimize the inner-cluster distances and maximize the intra-cluster distances. Prior knowledge about the dataset can be modelled as constraints. The simplest constraints are pairwise *must-link* and *cannot-link* constraints. Pairwise constraints can be either instance-based or metric-based. Instance-based constraints directly the clustering algorithm by enforcing constraints on their instance pairs. They can be hard or soft, allowing different degrees of constraint violation. Metric-based constraints map the constraints to the distance metric or similarity measure of the clustering algorithm. Pairwise constraints are limited to partitional clustering. So-called Must-Link-Before (MLB)

constraints introduce prior knowledge into hierarchical clustering. A MLB constraint

$$MLB_{xyz} = (d_x, d_y, d_z) \quad (1)$$

specifies an agglomerating order on three data points d_x, d_y, d_z . Symmetry applies for the first two elements. For the MLB constraint in Equation 1 both Definition 1 and Definition 2 must hold true. This means that the must-link constraint between the first and second item is stronger than that between the first and third item.

Definition 1 (MLB implication). *Given MLB constraint $MLB_{xyz} : \forall c \in H : d_x \in c \wedge d_z \in c \rightarrow d_y \in c$*

Definition 2 (MLB counter example). *Given MLB constraint $MLB_{xyz} : \exists c \in H : d_x \in c \wedge d_y \in c \wedge d_z \notin c$*

A MLB constraint for hierarchical clustering can be transformed into multiple pairwise constraints for partitional clustering by adding one must-link constraint between d_x and d_y and two cannot-links between d_x, d_z and d_y, d_z [?].

HAG and iHAG The hierarchical agglomerative clustering (HAG) algorithm iteratively merges the two most similar clusters into one. The instance-based hierarchical agglomerative clustering (iHAG) extends the HAG algorithm by not only considering similarity but also constraint violations [?].

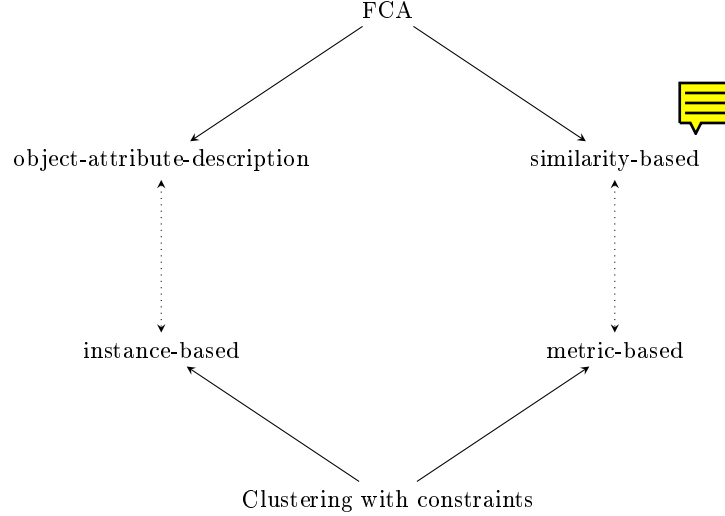
was heißt ihac ist besser als hac? Stefan

Attribute Exploration What is Attribute Exploration? [?]

Incomplete knowledge Formal Concept Analysis (FCA) with incomplete knowledge? [?]

Formal Concept Analysis (FCA) The mathematical theory of FCA [?] is a method to analyze relational data in a hierarchical structure. The triple $\mathbb{K} := (G, M, I)$ defines the *formal context*. G is the set of *objects*, M is the set of *attributes*, and $I \subseteq G \times M$ is the *incidence relation*, which describes when an object g has an attribute m . The object derivation $(\cdot)' : \mathcal{P}(G) \rightarrow \mathcal{P}(M)$ defines for any subset $A \subseteq G$ as $A' := \{m \in M \mid \forall g \in A : (g, m) \in I\}$. The attribute derivation $(\cdot)' : \mathcal{P}(M) \rightarrow \mathcal{P}(G)$ is defined for any subset $B \subseteq M$ as $B' := \{g \in G \mid \forall m \in B : (g, m) \in I\}$. The same notation $(\cdot)'$ is used for both derivation operators. A formal concept of $\mathbb{K}$ is a pair (A, B) where $A \subseteq G$ and $B \subseteq M$ satisfy the conditions $A' = B$ and $B' = A$. The set $\mathfrak{B}(\mathbb{K})$ denotes all formal concepts of $\mathbb{K}$. The concept lattice $\underline{\mathfrak{B}}(\mathbb{K}) := (\mathfrak{B}(\mathbb{K}), \leq)$ is defined by the set of all formal concepts with the order relation $(A, B) \leq (C, D)$ if and only if $A \subseteq C$.

Fig. 1: Similarities between FCA and constrained clustering. Similarity-based is 2D representation, where dimensions are not explicitly interpretable (unlike with object-attribute description). Distances in this space have a semantic meaning. (Stefan knows more)



4 Method

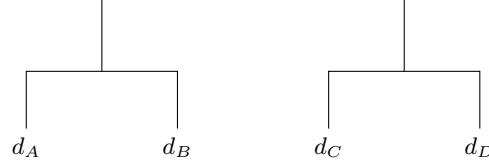
Implications are usually defined on attribute sets rather than on object sets [?], but we will work with object exploration since it aligns directly with constrained hierarchical clustering. The methodology proposed is directly applicable to both versions, differing only by the necessity of transposing the context for visualization via concept lattices. We denote the data points $\mathcal{D} = \{d_A, d_B, d_C, d_D, \dots\}$ the set of objects G and the clusters $c_i \in \mathcal{C}$ the set of attributes M . For simplicity, we explain our idea on the toy example given in Table 1.

Table 1: Formal context

	c_1	c_2	$c_1 \vee c_2$	$\dots$
d_A	$\times$		$\times$	
d_B	$\times$		$\times$	
d_C		$\times$	$\times$	
d_D		$\times$	$\times$	
$\vdots$				$\ddots$

The corresponding hierarchical cluster is given as a dendrogram in Figure 2. The bottom elements are data points in $\mathcal{D}$. Iterative merging of clusters are indicated by agglomerating lines from the bottom to the top.

Fig. 2: Dendrogram with objects on the bottom.



There are two directions to consider for proving the equivalence of both presentations. First, we will outline how a hierarchical cluster can be turned into its corresponding formal context and next, we will describe the other direction. In the following, we will describe the hierarchical cluster as a dendrogram.

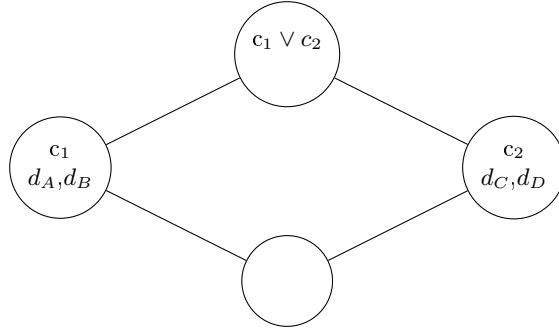
Hierarchical cluster to formal context. For each cluster on the lowest hierarchical level, i.e., the lowest line between leaf elements, add an attribute c_i with $i > |\mathcal{C}|$. Next, update the incidence relation $I = I \cup \{d | d \in c_i\}$, i.e., add each data point d of that cluster to the new attribute's extent, such that $(c_i)'$ contains all data points of that cluster. Once all lower clusters have been processed, proceed with the next higher cluster hierarchy. For each cluster emerging from sub-clusters $c_i, c_j \in \mathcal{C}$, add a new attribute $c_i \vee c_j$. This attribute's extent contains all attributes in either one of the sub-cluster's extents, i.e., $(c_i \vee c_j)' = (c_i)' \cup (c_j)'$. Proceed likewise for all clusters on that hierarchy and repeat the process for higher level emerging clusters.

Formal context to hierarchical cluster. Given a formal context (G, M, I) where G denotes the set of data points $\mathcal{D}$ and M denotes the set of clusters $c_i \in \mathcal{C}$, all objects $d \in G$ are leaf nodes of the dendrogram. For each attribute $a_i \in M$, identify any other attributes $a_j \in M$ whose extent is a superset of a_i 's extent. Save those attributes in a successor list (or better: set) s_{a_i} . To keep track of already processed cluster attributes, initialize a list *processed*. To keep track of the constraints add a list of resulting constraints r . Next, select one attribute a_i with minimal extent cardinality $|(a_i)'|$, i.e., the smallest cluster. For any pair of objects in a_i 's extent, i.e. data points $d_i, d_j \in (a_i)'$, add a pairwise **must-link** constraint to r . To capture the hierarchical relation between the clusters, for d_i, d_j and any object d_k in a_i 's successor list's items' extent excluding d_i, d_j , i.e., data point $d_k \in (a_i)' \setminus \{d_i, d_j\}$ for attribute $a \in s_{a_i}$, add a **MLB** constraint MLB_{d_i, d_j, d_k} . Once all MLB constraints for the pair d_i, d_j and objects $d_k \in (a_i)' \setminus \{d_i, d_j\}$ have been added, the next pair in $(a_i)'$ can be processed. Once all pairs in $(a_i)'$ have been processed, a_i is added to *processed*. Repeat the steps

for the next attribute a_j , i.e., cluster, with minimal extent cardinality $|(a_j)'|$ which has not yet been processed, i.e., is not element of *processed* is selected. The algorithm ends once all cluster attributes have been added to *processed*. The resulting set of MLB constraints models the solution space of possible clusters. Any clustering algorithm, i.e., including HAG and iHAG, that respects these *hard* constraints can produce a possible solution that lives within this solution space.

Formal context to concept lattice. Given a formal context (G, M, I) where G denotes the set of data points $\mathcal{D}$ and M denotes the set of clusters $c_i \in \mathcal{C}$, we can compute a meaningful concept lattice representation. The resulting concept lattice for the formal context in Table 1 is given in Figure 3.

Fig. 3: Concept lattice for the formal context.



5 **Dataset**

6 **Implementation**



7 **Case Study**

8 **Conclusion and Future Work**